AUTONATEAI SCREEPS LEAGUE

Tutorial 02: Target Selection

A practical local-server Screeps module **docs/tutorials/02-target-selection.md**

Episode 2: Two Sources, One Colony

Your collaborator was right, and it's a little smug about it. The moment `Harvester2` showed up, it walked to the exact same source as `Harvester1` and stood there waiting its turn — like two engineers both trying to deploy off the same branch at 4:59 on a Friday.

"That's not a bug," it says. "That's a script that never had to think about more than one worker. It's about to."

Two sources exist in this room. One creep knew that already and ignored it, because it never had competition. Two creeps don't get that luxury.

Goal

Get from the single-harvester loop in Tutorial 01 to two harvesters that split the room's sources instead of fighting over one, with:

- A second harvester spawned alongside the first
- A source-assignment rule that survives creep death and respawn
- Both creeps working in parallel from tick one

Prerequisites

Tutorial 01 is complete:

- `Spawn1` exists.
- `Harvester1` exists and the auto-respawn script from Step 6 is running.
- The room has at least two visible energy sources (this was a Step 1 checkpoint).

Step 1: See the Problem

Open the console and check how many sources this room actually has:

```
Game.spawns.Spawn1.room.find(FIND_SOURCES).length
```

Expected result: `2` (or more, depending on the room you chose).

Now look at the Tutorial 01 script again. `sources[0]` is hardcoded. Every creep that runs that code walks to the same source, every time, forever. That's fine for one creep. It's a waste of a room's income for two.

Step 2: Spawn a Second Harvester

In the console:

```
Game.spawns.Spawn1.spawnCreep([WORK, CARRY, MOVE], 'Harvester2');
```

Expected result: `0`.

Checkpoint:

- `Harvester2` appears next to `Harvester1`.
- Both creeps currently run the same Tutorial 01 script, so both head for `sources[0]`.

This is the bug made visible on purpose. Watch it happen once before fixing it.

> 📸 **Screenshot placeholder:** Both harvesters stacked on the same source tile, one visibly waiting its turn — the exact "two founders, one laptop" moment this step is designed to produce.

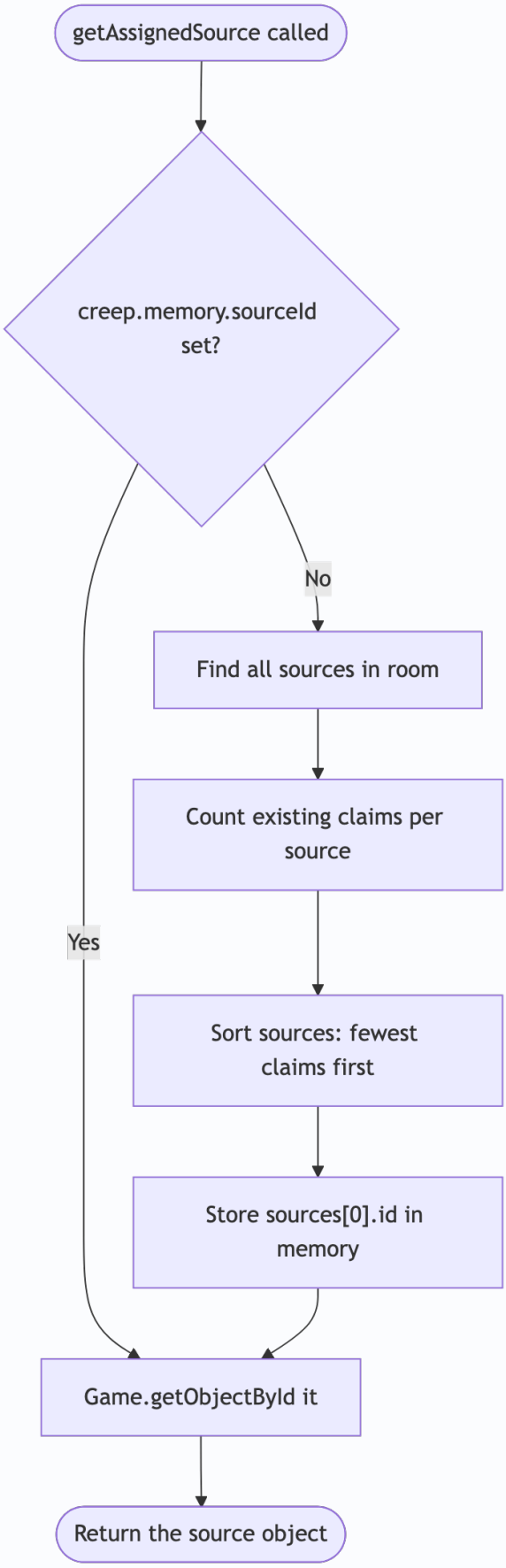
Step 3: Assign Sources Instead of Guessing

Replace `main` with a version that gives each creep a `sourceId` in Memory the first time it needs one, and reuses that assignment on every later tick:

```
module.exports.loop = function () {
  for (const name in Memory.creeps) {
    if (!Game.creeps[name]) {
      delete Memory.creeps[name];
    }
  }

  const harvesterNames = ['Harvester1', 'Harvester2'];
  const spawn = Game.spawns.Spawn1;

  for (const name of harvesterNames) {
    if (!Game.creeps[name] && !spawn.spawning) {
      spawn.spawnCreep([WORK, CARRY, MOVE], name);
    }
  }

  for (const name of harvesterNames) {
    const creep = Game.creeps[name];
    if (!creep) {
      continue;
    }
    runHarvester(creep);
  }
};

function runHarvester(creep) {
  if (creep.store.getFreeCapacity() > 0) {
    const source = getAssignedSource(creep);
    if (creep.harvest(source) === ERR_NOT_IN_RANGE) {
      creep.moveTo(source, { visualizePathStyle: { stroke: '#ffaa00' } });
    }
    return;
  }

  const spawn = Game.spawns.Spawn1;
  if (creep.transfer(spawn, RESOURCE_ENERGY) === ERR_NOT_IN_RANGE) {
    creep.moveTo(spawn, { visualizePathStyle: { stroke: '#ffffff' } });
  }
}

function getAssignedSource(creep) {
  if (!creep.memory.sourceId) {
    const sources = creep.room.find(FIND_SOURCES);
    const claimCounts = {};
    sources.forEach((source) => {
      claimCounts[source.id] = 0;
    });

    for (const name in Game.creeps) {
      const assignedId = Game.creeps[name].memory.sourceId;
      if (assignedId && assignedId in claimCounts) {
        claimCounts[assignedId] += 1;
      }
```

```
    }

    sources.sort((a, b) => claimCounts[a.id] - claimCounts[b.id]);
    creep.memory.sourceId = sources[0].id;
  }

  return Game.getObjectById(creep.memory.sourceId);
}
```

`getAssignedSource` in one picture – this only runs the expensive part once per creep, ever, not every tick:


getAssignedSource called
creep.memory.sourceId set?
No
Find all sources in room
Count existing claims per source
Yes
Sort sources: fewest claims first
Store sources[0].id in memory
Game.getObjectById it
Return the source object

Save the script.
Checkpoint:

- `Harvester1` keeps working the source it already had (or gets reassigned to whichever is least crowded, if you reset memory).
- `Harvester2` picks the *other* source, not the same one.
- Both creeps deliver to `Spawn1` independently.

Step 4: Understand Why This Survives Respawn

`getAssignedSource` only writes `creep.memory.sourceId` the first time it's missing. After that, the assignment persists in `Memory.creeps[name]` for the creep's whole lifetime — even across ticks where the room is out of view.

When a creep dies, `Memory.creeps[name]` gets cleaned up by the loop at the top of `main` (the same cleanup from Tutorial 01, Step 6). The replacement creep spawned under that name starts with no memory, so it re-runs the assignment logic and picks whichever source currently has the fewest claims.
Checkpoint:

```
Game.creeps.Harvester1.memory
```

Expected result: an object containing a `sourceId` that matches one of the two source IDs from `creep.room.find(FIND_SOURCES)`.

Step 5: Force a Reassignment Test

Kill one harvester on purpose and watch the fix hold:

```
Game.creeps.Harvester2.suicide();
```

Wait for the auto-respawn logic to create a new `Harvester2`.

Checkpoint:

- The new `Harvester2` gets a fresh `sourceId` the moment it needs one.
- It does not necessarily pick the same source it had before — it picks whichever source has the fewest active claims *right now*.

Troubleshooting

If both creeps still pile on one source:

```
Game.creeps.Harvester1.memory.sourceId
Game.creeps.Harvester2.memory.sourceId
```

If these are identical, check that `getAssignedSource` is reading `Game.creeps` (all creeps in the room) and not just the current creep — the claim count has to see both creeps to balance them.

If a creep's source is `null` after `Game.getObjectById`, the ID it stored no longer matches a valid object. This shouldn't happen with sources (they don't expire), but if you see it, log `creep.memory.sourceId` and compare against `creep.room.find(FIND_SOURCES).map(s => s.id)`.

Completion Criteria

You are done when:

- Two harvesters exist and both are alive at the same time.
- Each is assigned a different source (in a room with two or more sources).
- Killing a creep and letting it respawn does not break the assignment logic.
- Dead creep memory is still cleaned up every tick.

Learning Notes

After completing the tutorial, write down:

- How many ticks did it take for both harvesters to be working in parallel?
- What would happen if this room only had one source and you spawned a third harvester?
- What's the difference between "assign once and remember" and "recompute the best source every tick"? Which one did you just build, and why does it matter for CPU?
- What should a builder or upgrader do differently from a harvester when picking a target?

Next: Episode 3 — Job Descriptions

Two harvesters doing the same job is manageable. It won't stay that way.

"Next time you spawn a creep," your collaborator says, "it's not going to be another harvester. It's going to have a different job entirely — and this script has no idea how to tell the difference. Right now your whole org chart is one job title."

`runHarvester` is the only behavior this colony knows. Episode 3 gives it more than one.

See `docs/roadmap.md` for the full season.